

Research Proposal: Graph Topologies and Incentive Stability in Multi-Agent Code Generation

Name: Nana Ye, Charlotte Ma, Jason Liu

Introduction, Prior Works & Motivation

Multi-agent LLM systems have recently shown strong gains in code generation by decomposing software development into specialized roles and introducing structured feedback. Frameworks such as MetaGPT and AutoGen formalize role-based coordination, while iterative approaches like AgentCoder, Reflexion, and MapCoder demonstrate that closed-loop testing and reflection can substantially improve correctness on benchmarks such as Mostly Basic Programming Problems (MBPP) (Hong et al., 2023; Wu et al., 2024; Huang et al., 2023; Shinn et al., 2023; Islam et al., 2024; Austin et al., 2021). However, existing systems are typically evaluated as holistic pipelines in which many factors change simultaneously: agent roles, prompts, tools, and stopping rules. As a result, it remains unclear how much of the observed performance gain arises specifically from the interaction graph structure. This lack of controlled analysis limits our understanding of how different topologies shape agents' incentives to spend tokens on verification under constrained budgets. As AI usage in software engineering scales in the near-future, inefficient multi-agent systems could lead to massively wasted compute costs.

This project evaluates how these graph structures affect the tradeoff between accuracy and compute cost on the MBPP benchmark using a fixed model (GPT-4o) (Austin et al., 2021). Specifically, we investigate whether complex graph structures discourage high-effort verification from agents due to varying incentives.

We analyze multi-agent interactions through the lens of Cooperative Game Theory, where the successful completion of a coding task (pass@1) is treated as a payoff generated by a team of agents. We utilize the Shapley value to provide an axiomatic, fair allocation of success by measuring the marginal contributions of each agent role within specific graphs (Shapley, 1953). Furthermore, we apply a utility-based framework to study incentive-stability. We define utility as the balance between task success and cost of tokens of effort, and we use it to determine whether a graph structure supports an approximate Nash Equilibrium where all agents are incentivized to perform high effort verification under constrained compute costs (Shoham & Leyton-Brown, 2008).

Problem Statement

Agents. Planner (PL), Coder (C), Reviewer (R), and Tester (T)

Actions. Each agent i selects an effort level $e_i \in \{0, 1\}$, where higher effort corresponds to more rigorous reasoning or verification and incurs a cost $C_i(e_i)$ measured by token usage and interaction rounds.

Outcome. For task x and graph g , the protocol runs for at most K rounds and terminates with $Y \in \{\text{pass}, \text{fail}\}$ based on unit-test execution.

Utility. Each agent’s utility is $U_i = V \cdot 1[Y = \text{pass}] - \lambda \cdot \text{tokens}_i$, where $V > 0$ is the value of success and $\lambda > 0$ scales compute cost. Graphs are compared by pass@1, total tokens, and whether the high-effort profile $e^* = (1, 1, 1, 1)$ is an approximate Nash equilibrium.

We want to investigate whether there exists a graph configuration such that for every agent and some reasonably small ϵ , such that $U_i(e=1, e^*) \geq U_i(e=0, e^*) + \epsilon$

Methodology

Graph configurations: We compare interaction graphs that differ only in topology and stopping rules, holding the model, tasks, and role prompts fixed:

- G0: Single-Agent Baseline. One-shot solution, optional self-check.
- G1: Waterfall Pipeline. Sequential $P \rightarrow C \rightarrow R \rightarrow T$ with minimal feedback.
- G2: AgentCoder-Style Loop. Iterative code–test–revise until tests pass or budget is exhausted.
- G3: MapCoder-Style Cycle. Retrieval–planning–coding–debugging under a fixed budget.

All graphs are implemented in a common orchestration framework (e.g., LangGraph); no fine-tuning or GPU resources are used.

Benchmark and metrics: Evaluation is conducted on MBPP. We report pass@1, total tokens, interaction rounds, and the stage where errors are first detected (review vs. testing).

Incentive stability: For each graph, we perform an empirical best-response test by forcing one role to low effort while others remain high and estimating the utility change ΔU_i . A graph supports stable high-effort verification if no role benefits from unilateral deviation.

Shapley attribution: Let $vg(S)$ denote expected pass@1 when only roles S are active. Shapley values are estimated via permutation sampling, $\phi_i(g) = E_{\pi} [vg(S_{\pi} \cup \{i\}) - vg(S_{\pi})]$, to quantify each role’s marginal contribution under different graphs.

Technical Challenges: Several challenges still exist for our study particularly on computational cost and effort manipulation. First, estimating Shapley values requires evaluating many subsets of agents, which can be expensive even with a small number of roles. We could potentially address this by using permutation-based Monte Carlo estimation and limiting evaluation to a representative subset of MBPP tasks. Second, because we do not fine-tune models, effort levels must be operationalized through prompt constraints, token budgets, and stopping rules. Ensuring that “low effort” and “high effort” conditions are comparable across graphs is nontrivial and would require prompt and budget calibration.

Reference

- Austin, J., Odena, A., Nye, M., Bosma, M., Michalewski, H., Dohan, D., Jiang, E., Cai, C. J., Terry, M., Le, Q. V., & Sutton, C. (2021). Program synthesis with large language models. <https://doi.org/10.48550/arXiv.2108.07732>
- Hong, S., Zhuge, M., Chen, J., Zheng, X., Cheng, Y., Wang, J., ... & Schmidhuber, J. (2023, August). MetaGPT: Meta programming for a multi-agent collaborative framework. In The twelfth international conference on learning representations.
- Huang, D., Bu, Q., Zhang, J. M., Luck, M., Qing, Y., & Cui, H. (2023). AgentCoder: Multi-agent-based code generation with iterative testing and optimisation. <https://doi.org/10.48550/arXiv.2312.13010>
- Islam, M. A., Ali, M. E., & Parvez, M. R. (2024). MapCoder: Multi-agent code generation for competitive problem solving. In Proceedings of ACL 2024 (Long Papers) (pp. 4912–4944). <https://doi.org/10.18653/v1/2024.acl-long.269>
- Shapley, L. S. (1953). A value for n-person games. In H. W. Kuhn & A. W. Tucker (Eds.), Contributions to the theory of games II (pp. 307–317). Princeton University Press. <https://doi.org/10.1515/9781400881970-018>
- Shinn, N., Cassano, F., Gopinath, A., Narasimhan, K., & Yao, S. (2023). Reflexion: Language agents with verbal reinforcement learning. Advances in Neural Information Processing Systems, 36, 8634-8652.
- Shoham, Y., & Leyton-Brown, K. (2008). Multiagent systems: Algorithmic, game-theoretic, and logical foundations. Cambridge University Press
- Wu, Q., Bansal, G., Zhang, J., Wu, Y., Li, B., Zhu, E., ... & Wang, C. (2024, August). Autogen: Enabling next-gen LLM applications via multi-agent conversations. In First Conference on Language Modeling.

Introduction

Multi-agent coding systems improve reliability by decomposing software development into specialized roles and introducing feedback loops. However, the efficacy of these systems is highly dependent on their interaction graph structure. While linear pipelines are simple, iterative test loops and multi-stage cycles alter mechanisms for information flow, error discovery, and incentives to spend tokens on verification. This project evaluates how these graph structures affect the tradeoff between accuracy and compute cost on the MBPP benchmark using a fixed model (GPT-4o). Specifically, we investigate whether complex graph structures discourage high-effort verification from agents due to varying incentives.

Theoretical Context

We analyze multi-agent interactions through the lens of Cooperative Game Theory, where the successful completion of a coding task (pass@1) is treated as a payoff generated by a team of agents. We utilize the Shapley value to provide an axiomatic, fair allocation of success by measuring the marginal contributions of each agent role within specific graphs [4]. Furthermore, we apply a utility-based framework to study incentive-stability. We define utility as the balance between task success and cost of tokens of effort, and we use it to determine whether a graph structure supports an approximate Nash Equilibrium where all agents are incentivized to perform high effort verification under constrained compute costs [5].

Previous Work

Early work on code generation studied whether large language models can synthesize short programs from natural-language descriptions, and introduced benchmarks to measure functional correctness. Austin et al. (2021) proposed the Mostly Basic Programming Problems (MBPP) dataset (974 Python tasks) and used it to evaluate program synthesis performance under few-shot prompting and fine-tuning. MBPP-style unit-test evaluation is now a standard way to compare code-generation methods.

A major recent trend is to decompose software development into specialized agent roles (e.g., planner/architect/engineer/tester) and orchestrate them with explicit workflows. MetaGPT formalizes this idea by encoding “standard operating procedures” (SOPs) into prompt sequences, aiming to reduce cascading errors when chaining multiple LLM steps (Hong et al., 2023). Frameworks such as AutoGen generalize this direction by providing infrastructure to define multi-agent conversation patterns and integrate tools/humans, treating interaction patterns as configurable components of a broader agent system (Wu et al., 2024). These systems demonstrate that multi-agent decomposition can improve reliability, but they are typically evaluated as *end-to-end packages*, where multiple design choices (roles, prompts, tools, and stopping rules) change at once.

Another strong line of work improves code generation through closed-loop iteration: generate code, execute tests (or use other feedback), and revise. AgentCoder introduces explicit roles for

a programmer, test designer, and test executor, using iterative testing feedback to guide refinement. In a related spirit (Huang et al., 2023), Reflexion shows that language agents can incorporate feedback through “reflection” text that conditions future attempts, improving performance across tasks without updating model weights (Shinn et al., 2023). Beyond simple code–test loops, MapCoder proposes a multi-agent program-synthesis cycle with agents for recalling examples, planning, coding, and debugging, aiming to mirror how humans solve programming problems in stages (Islam et al, 2024). These approaches provide strong evidence that iteration and structured stages can improve correctness, but they often do not isolate the *interaction topology itself* (pipeline vs. loop vs. richer cycle) as the primary variable under controlled conditions.

These works provide compelling demonstrations that role specification, execution feedback, and richer agent tooling can improve coding outcomes. Yet there remains limited controlled evidence about how much of the benefit comes specifically from the interaction graph structure, and limited analysis of role contribution and incentive effects under compute costs. This motivates our project’s focus on systematically comparing representative topologies under fixed model and benchmark conditions, then analyzing contribution and verification incentives using cooperative-game and utility-based lenses.